



CLOUD ENGINEER COMMAND-LINE TOOLKIT

Essential Commands for Real Cloud Engineering Work



AWS



Azure



Google Cloud



Linux & CLI

- ✓ Commands
- ✓ Examples
- ✓ Troubleshooting
- ✓ Real Scenarios
- ✓ Best Practices



Same
Commands
Bigger
Opportunities



PRACTICAL
EXAMPLES



REAL-WORLD
SCENARIOS



TROUBLESHOOTING
GUIDES



BEST
PRACTICES



BUILDS
JOB-READY SKILLS

LET'S BUILD YOUR CLOUD CAREER

VERIQTA

LEARN | PRACTICE | BUILD | GROW

The Cloud Engineer's Terminal

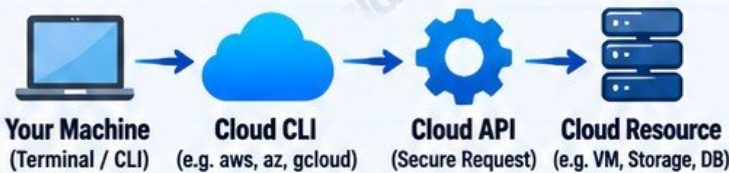
1. What a Cloud Engineer Actually Does From the Command Line

- ✓ Manage cloud resources
- ✓ Deploy and update applications
- ✓ Configure and troubleshoot infrastructure
- ✓ Automate repetitive tasks
- ✓ Monitor system health and logs
- ✓ Investigate and fix issues
- ✓ Work with APIs and cloud services

2. Key Terms You Should Know

Terminal	A program where you type commands (e.g. Terminal on Mac, PowerShell, Command Prompt).
Shell	The command interpreter (e.g. bash, zsh, fish).
CLI	Command-Line Interface. A tool to interact with cloud services (e.g. aws, az, gcloud).
API	Application Programming Interface. Used by CLIs to communicate with cloud services.
SDK	Software Development Kit. Libraries in code (e.g. Python, Java) to use cloud APIs.
Console	Web interface (browser) to manage resources. Useful, but limited for automation.

3. How the Command Line Connects to the Cloud



4. Why Engineers Automate Instead of Only Using Web Consoles

- ✓ Faster and more efficient
- ✓ Repeatable and consistent
- ✓ Manage many resources at once
- ✓ Can be integrated into scripts and CI/CD
- ✓ Reduces human error
- ✓ Works without a browser (servers, pipelines, etc.)
- ✓ Essential for real-world cloud operations

5. Useful Commands to Try

COMMAND	WHAT IT DOES	EXAMPLE OUTPUT
whoami	Shows your current username.	clouduser
pwd	Shows your current working directory.	/home/clouduser
echo \$SHELL	Shows the shell you are using.	/bin/bash



Remember

- Commands can change real resources.
- Start with read-only commands when unsure.
- Use --help to learn about any command.
- Practice in a safe environment (e.g. a test account).



"Junior Engineer Tip"

Understand what a command changes before running it."

Always read and think before you execute, especially in cloud environments.

**SMALL COMMANDS
POWER BIG CAREERS**

Navigate Linux Like a Cloud Engineer

1. Files and Directories

- Files store data (e.g. text, config, code).
- Directories (folders) organise files.
- Linux uses a tree structure starting from / (root).
- Each file and directory has a path.

2. Types of Paths

- Absolute path:** Full path from root.
Example: `/home/clouduser/project`
- Relative path:** Path relative to your current directory.
Example: `project/config`
- Use absolute paths when you need to be exact. Use relative paths for shorter navigation.

3. Home Directory vs Working Directory



Home Directory

- Your personal folder.
- Shortcut: `~`
- Example: `/home/clouduser`



Working Directory

- The directory you are currently in.
- Shows with: `pwd`
- Changes when you use `cd`
- Example: `/home/clouduser/project`

4. Useful Commands

COMMAND	WHAT IT DOES
<code>pwd</code>	Shows your current working directory.
<code>ls</code>	Lists files and directories.
<code>ls -lah</code>	Lists in long format (permissions, size, hidden files).
<code>cd <dir></code>	Changes directory.
<code>tree</code>	Shows directory structure (install with: <code>sudo apt install tree</code>).

5. Example Usage

```
clouduser:~$ pwd
/home/clouduser
clouduser:~$ ls
project scripts README.md
clouduser:~$ ls -lah
total 24
drwxr-xr-x 5 clouduser clouduser 4096 ...
clouduser:~$ cd project
clouduser:~/project$ pwd
/home/clouduser/project
clouduser:~/project$ tree
.
├── main.tf
├── variables.tf
├── modules
└── networking.tf
```

6. Hidden Configuration Directories

Cloud tools store configuration files in hidden directories (names start with `.`).

```
clouduser:~$ ls -lah | grep "^d"
drwx----- 3 clouduser clouduser 4096 .aws
drwx----- 3 clouduser clouduser 4096 .config
drwx----- 2 clouduser clouduser 4096 .ssh
```

Examples:

- `~/.aws` (AWS CLI configuration)
- `~/.azure` (Azure CLI configuration)
- `~/.gcloud` (Google Cloud configuration)
- `~/.ssh` (SSH keys)

7. Common Failure



Running a command from the wrong directory.

- You might get "file not found" errors.
- You might modify the wrong file.
- Always check your location with `pwd` before running important commands.

8. Practice: Navigate a Small Infrastructure Project

- Create a project folder: `mkdir infra-project`
- Navigate into it: `cd infra-project`
- Create subdirectories: `mkdir modules scripts configs`
- Use `tree` to view the structure.
- Move between directories using `cd`, `..` and absolute paths.

Inspect and Manipulate Files



Cloud engineers constantly work with YAML, JSON, logs, keys, and configuration files. These commands help you view, create, copy, move and manage files and directories.

Files
Configs
Logs
Keys
Everything
Matters!

1. COMMON FILE COMMANDS

COMMAND	WHAT IT DOES	WHEN TO USE IT	EXAMPLE
cat	Shows file content in the terminal.	Use for small files.	<code>cat file.txt</code>
less	Views file content page by page.	Use for large files (safer).	<code>less /var/log/app.log</code>
head	Shows the first lines of a file.	Quickly check the beginning of a file.	<code>head -n 10 file.txt</code>
tail	Shows the last lines of a file.	Check recent logs or updates.	<code>tail -n 10 app.log</code>
touch	Creates an empty file or updates timestamp.	Create new files quickly.	<code>touch newfile.txt</code>
cp	Copies files or directories.	Backup or duplicate files.	<code>cp file.txt backup.txt</code>
mv	Moves or renames files or directories.	Organize or rename files.	<code>mv file.txt config.txt</code>
mkdir	Creates a new directory.	Create folders for projects.	<code>mkdir project</code>

2. WHY USE less?

- ✓ Large files (logs, configs) can be thousands of lines.
- ✓ less lets you scroll, search and exit easily.
- ✓ It does not flood your terminal with output.
- ✓ You can search using `/keyword`.
- ✓ Press `q` to exit.

```
clouduser:~$ less app.log
(use /error to search)
Press q to exit
```

3. FILE TYPES YOU WILL WORK WITH

				
YAML	JSON	LOG	KEY	CONF
.yaml .yml	.json	.log	.pem .key	.conf .cfg .env

4. DANGEROUS OPERATIONS (BE CAREFUL)



rm file.txt

- Permanently deletes a file.
- No confirmation, no recycle bin.

rm -r directory/

- Recursively deletes a directory and all its files.
- This cannot be easily undone.

Always double-check the path before running **rm** commands.
Make sure you are in the correct directory.

5. PRACTICAL EXAMPLE

```
clouduser:~/project$ mkdir infra
clouduser:~/project$ cd infra
clouduser:~/project/infra$ touch main.yaml
clouduser:~/project/infra$ ls
main.yaml
clouduser:~/project/infra$ cp main.yaml backup.yaml
clouduser:~/project/infra$ mv backup.yaml config.yaml
clouduser:~/project/infra$ ls
config.yaml main.yaml
```

- ✓ You just created a folder, created a file, copied it, renamed it and listed the contents.

6. JUNIOR ENGINEER TIP



Understand what a command changes before running it. When in doubt, check your current directory with `pwd` and list files with `ls` before making changes.

7. REMEMBER

- ✓ Use **less** for large files.
- ✓ Be careful with **rm** and **rm -r**.
- ✓ Check your current location with `pwd`.
- ✓ Keep your files organised in directories.
- ✓ Practice these commands in a safe environment.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Search Files, Logs, and Configuration

Find What You Need. Faster.



Finding evidence instead of manually opening files saves time and helps you troubleshoot like a real engineer.

1. Essential Search Commands

COMMAND	WHAT IT DOES	WHEN TO USE IT
<code>grep PATTERN file</code>	Searches for a pattern in a file.	Find specific text in a file (e.g. ERROR in a log).
<code>grep -R PATTERN dir</code>	Searches recursively in a directory.	Search in all files inside a directory.
<code>find PATH -name "file"</code>	Finds files by name.	Locate files (e.g. .yaml, .conf) in directories.
<code>wc file</code>	Counts lines, words or bytes.	Check file size or number of lines.
<code>sort file</code>	Sorts lines.	Organize output (alphabetically/numerically).
<code>uniq file</code>	Removes duplicate adjacent lines.	Clean up repeated output (usually with sort).

4. Useful Command Combinations

<code>grep -R "error" .</code>	Search for "error" in all files in current directory.
<code>find . -name "*.yaml" wc -l</code>	Count how many .yaml files.
<code>cat app.log grep ERROR wc -l</code>	Count ERROR lines in a log file.
<code>sort file.txt uniq</code>	Sort and remove duplicate lines.
<code>grep -i "warning" app.log tail</code>	Show last warning entries (case-insensitive).

5. Practical Use Cases

- ✓ Search configuration files for specific values.
- ✓ Find all files related to a service.
- ✓ Investigate logs for errors or warnings.
- ✓ Locate infrastructure files (e.g. .yaml, .tf, .json).
- ✓ Count and analyze output.

2. Pipes (|) – Combine Commands

- The pipe (|) sends the output of one command as input to another command.
- Helps you filter, sort, count and process data.
- Used heavily in real DevOps and cloud work.

```
clouduser:~$ cat app.log | grep ERROR | wc -l
25 # Number of ERROR lines
```

3. Real-World Examples



Search logs for ERROR

```
clouduser:~$ grep "ERROR" app.log
2025-09-10 ERROR Failed to connect
2025-09-10 ERROR Timeout occurred
2025-09-10 ERROR Invalid token
```



Locate every .yaml file

```
clouduser:~$ find . -name "*.yaml"
./k8s/deployment.yaml
./k8s/service.yaml
./config/app.yaml
```

6. Troubleshooting Tip



“Search for evidence before changing configuration.”

- ✓ Check logs.
- ✓ Find related files.
- ✓ Understand what is happening.
- ✓ Only make changes after you have evidence.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Users, Permissions, and sudo

Control Access. Keep Your Systems Secure.

1. Users, Groups, and Owners

- **User:** A person or application that can access the system.
- **Group:** A collection of users with the same permissions.
- **Owner:** The user who owns a file or directory.
- These control who can read, write, or execute files and directories.

3. Understanding File Permissions

PERMISSION	SYMBOL	MEANING
Read	r	View file content (or list directory).
Write	w	Modify file content (or create/delete files).
Execute	x	Run the file (or enter directory).

Permission Format (e.g. -rwxr-xr--)



2. Useful Commands

COMMAND	WHAT IT DOES
whoami	Shows your current username.
id	Shows your user ID (UID) and group ID (GID).
groups	Shows the groups you belong to.
ls -l	Displays file permissions, owner and group.
chmod	Changes file or directory permissions.
chown	Changes the owner (and group) of a file or directory.
sudo	Runs a command as another user (usually root).

4. Examples

```
clouduser:~$ whoami
clouduser
clouduser:~$ id
uid=1000(clouduser) gid=1000(clouduser)
clouduser:~$ groups
clouduser sudo docker
clouduser:~$ ls -l app.sh
-rwxr-xr-- 1 clouduser clouduser 1250 Sep 10 10:00 app.sh
clouduser:~$ chmod 755 app.sh
clouduser:~$ chown clouduser:devops app.sh
clouduser:~$ sudo systemctl restart nginx
```

5. Why chmod 777 is Usually the Wrong Fix



- **chmod 777** gives full read, write and execute access to everyone.
- It can expose sensitive files.
- It increases the risk of accidental or malicious changes.
- Use the minimum permissions required instead (least privilege).

6. Cloud Connection



Permissions are critical in cloud environments:

- SSH private keys should be 600 (read/write for owner only).
- Configuration files (e.g. ~/.aws) should not be world-readable.
- The correct file permissions prevent unauthorized access to your cloud credentials.

7. Security Reminder



"Use the principle of least privilege. Give only the permissions that are required, no more, no less."

8. Quick Practice

- ✓ Check your current user and groups.
- ✓ View permissions of files in your home directory.
- ✓ Change permissions on a test file.
- ✓ Try running a command with sudo (e.g. **sudo ls /root**).



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Processes, Services, and System State

Learn
Practice
Build
Grow

Observe. Diagnose. Keep Things Running.

1. Process vs Service



Process

- A running program in memory.
- Has a PID (Process ID).
- Can be started, stopped and killed.
- Examples: nginx, python, java, bash.



Service

- A managed process (usually via systemd).
- Starts at boot (if enabled).
- Can be restarted automatically.
- Examples: nginx.service, sshd.service, docker.service.

2. Useful Commands

COMMAND	WHAT IT DOES
ps	Shows running processes.
top	Real-time view of system processes.
pgrep	Finds process ID by name.
systemctl	Manages services (start, stop, status, etc).
journalctl	Reads service logs (systemd).

3. Find Which Process is Running

```
clouduser:~$ ps aux | grep nginx
root      1253  0.0  0.1 6284 2320 ?        Ss   10:01 nginx
clouduser 2345  0.0  0.0 4340  980 pts/0    S+   10:05 grep  nginx

clouduser:~$ pgrep -l nginx
1253 nginx
```

4. Check Whether a Service Failed

```
clouduser:~$ systemctl status nginx
● nginx.service - A high performance web server
   Loaded: loaded (/lib/systemd/system/nginx.service)
   Active: failed (Result: exit-code)
     Docs: man:nginx(8)
    Main PID: 1253 (code=exited, status=1/FAILURE)
```

5. Read Service Logs

```
clouduser:~$ journalctl -u nginx -n 20
Sep 10 10:01:23 server nginx[1253]: Starting nginx...
Sep 10 10:01:23 server nginx[1253]: [emerg] bind() to
Sep 10 10:01:23 server nginx[1253]: failed (98:
Sep 10 10:01:23 server systemd[1]: nginx.service:
Failed with result 'exit-code'.
```

6. Common Failure



Application installed but service not running.

- Package is installed but service is not started.
- Service is not enabled at boot.
- Configuration error prevents the service from starting.
- Port already in use.
- Missing dependencies (e.g. database, network).

7. Troubleshooting Flow



8. Quick Tips

- ✓ Use ps, top or pgrep to find running processes.
- ✓ Always check service status and logs when something is not working.
- ✓ Understand error messages, they usually tell you the real cause.
- ✓ Fix the root cause instead of just restarting.
- ✓ Enable services that should start at boot: `systemctl enable <service>`
- ✓ After fixing, always verify that the service is running and healthy.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Networking Commands Every Cloud Engineer Needs

Inspect. Diagnose. Connect.

Learn
Practice
Build
Grow

1. Key Concepts

- **IP Address:** Identifies a device on a network.
- **Interface:** Network connection (e.g. eth0, ens5).
- **Route:** Path to reach other networks.
- **Port:** Entry point for network traffic (e.g. 22, 80).

2. Useful Commands

COMMAND	WHAT IT REVEALS
<code>ip addr</code>	Shows network interfaces and IP addresses.
<code>ip route</code>	Shows routing table (default route, gateways).
<code>ss -tulpn</code>	Shows open ports and listening services.
<code>hostname</code>	Shows your system hostname.

3. View IP Addresses and Interfaces

```
clouduser:~$ ip addr
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
   inet 10.0.0.12/24 brd 10.0.0.255 scope global eth0
   inet6 fe80::a00:27ff:fe4e:66a1/64 scope link
3: lo: <LOOPBACK,UP,LOWER_UP>
   inet 127.0.0.1/8 scope host lo
```

- Shows all network interfaces.
- Look for the **inet** line to find your IP address.
- Common interface names: eth0, ens5, enp0s3.

4. View Routing Table

```
clouduser:~$ ip route
default via 10.0.0.1 dev eth0
10.0.0.0/24 dev eth0 proto kernel scope link
169.254.0.0/16 dev eth0 scope link
```

- Shows how your system reaches other networks.
- The default route (default via) is usually your gateway.
- In the cloud, this connects to your VPC/subnet route table.

5. Check Open Ports and Listening Services

```
clouduser:~$ ss -tulpn
Netid  State  Local Address:Port  Process
tcp    LISTEN  0.0.0.0:22         1234/sshd
tcp    LISTEN  0.0.0.0:80         2345/nginx
tcp    LISTEN  [::]:22          1234/sshd
```

- Shows open ports, listening services and their process IDs.
- Helps you verify if a service is running (e.g. SSH, Nginx).
- Useful for troubleshooting connectivity issues.

6. Check Hostname

```
clouduser:~$ hostname
ip-10-0-0-12
clouduser:~$ hostname -f
ip-10-0-0-12.ec2.internal
```

- Shows your system hostname.
- Helpful to confirm which server you are on.
- In cloud environments, hostnames often include instance or internal DNS names.

7. Private vs Public IP



Private IP

- Used inside a private network (e.g. VPC).
- Common ranges: 10.x.x.x, 172.16.x.x, 192.168.x.x.
- Not directly accessible from the internet.



Public IP

- Used to access a server from the internet.
- Assigned by your cloud provider (e.g. AWS, Azure, GCP).
- Can change (especially for non-static IPs).

8. Cloud Connection



- **VMs:** Have network interfaces with private/public IPs.
- **Subnets:** Define IP ranges for your resources.
- **Routing Tables:** Control network traffic (default routes).
- **Security Groups:** Act as firewalls (allow/deny ports).
- These CLI commands help you verify network configuration and troubleshoot connectivity.

9. Practice



- ✓ Run `ip addr` and identify your main interface and IP address.
- ✓ Run `ip route` and find your default gateway.
- ✓ Run `ss -tulpn` and check which ports are open.
- ✓ Run `hostname` and note your server name.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Test Connectivity with ping and traceroute

Diagnose the Path. Find the Problem.

1. Key Points

- Connectivity is not one problem.
- You can reach some services and not others.
- A network can fail at multiple points.
- Use **ping** and **traceroute** to understand where a problem is happening.

2. Useful Commands

COMMAND	WHAT IT DOES
ping <host>	Checks basic network reachability (using ICMP).
traceroute <host>	Shows the path your traffic takes (to find where it stops).
tracepath <host>	Alternative to traceroute (often available on Linux).

3. Successful ping Result (Example)

```
clouduser:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=14.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=13.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=14.1 ms
```

- ✓ You receive replies from the destination.
- ✓ This means network connectivity is working.
- ✓ Shows latency (time) and TTL (hops remaining).

4. Failed ping Result (Example)

```
clouduser:~$ ping 10.0.1.10
PING 10.0.1.10 (10.0.1.10) 56(84) bytes of data.
--- 10.0.1.10 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss
```

- ✗ No replies are received.
- ✗ This does not always mean the server is down.
- ✗ It could be a firewall blocking ICMP.
- ✗ Many cloud environments block ping by default.

5. traceroute Result (Example)

```
clouduser:~$ traceroute example.com
traceroute to example.com (93.184.216.34), 30 hops max
 1  10.0.0.1 (10.0.0.1)  1.234 ms  1.102 ms  1.098 ms
 2  172.16.1.1 (172.16.1.1)  5.423 ms  5.381 ms  5.390 ms
 3  203.0.113.1 (203.0.113.1)  12.531 ms  12.488 ms  12.450 ms
 4  93.184.216.34 (93.184.216.34)  20.114 ms  20.076 ms  20.102 ms
```

- Shows each hop (router) between your machine and the destination.
- Helps identify where the connection stops or becomes slow.
- Useful for network troubleshooting.

6. What traceroute Tells You

-  Each hop IP address.
-  Response time at each hop.
-  Where the connection stops (if it fails).
-  Whether the destination is reachable.
-  Helps you identify if the issue is local, in the cloud, or at the destination.

7. Why a Failed ping Does Not Mean the Server is Down

- 💡 Firewalls may block ICMP traffic (ping).
- 💡 Many cloud providers disable ping by default.
- 💡 The server can still be running and reachable via SSH, HTTP, or other ports.
- 💡 Always test other methods (e.g. curl, telnet, nc).

8. Troubleshooting Model



9. Practice

- ✓ Run ping to a public IP (e.g. 8.8.8.8).
- ✓ Run **traceroute** or **tracepath** to a domain (e.g. example.com).
- ✓ Identify your network path and note where latency increases.

Remember

 "No response is a response. It tells you something."

DNS Troubleshooting Toolkit

From Name to IP. Find the Problem. Fix the Connection.

1. DNS Lookup Flow



- You use a domain name (e.g. example.com).
- DNS translates the name into an IP address.
- Your machine then connects to the IP address.
- If DNS fails, the application cannot be reached.

2. Useful Commands

COMMAND	WHAT IT DOES
dig <domain>	Detailed DNS query. Shows A, AAAA, CNAME, MX and more.
nslookup <domain>	Simple DNS lookup. Shows the resolved IP.
host <domain>	Another DNS lookup tool. Clean and easy to read.

3. Example: Successful DNS Lookup

```

clouduser:~$ dig example.com +short
93.184.216.34
93.184.216.35

clouduser:~$ nslookup example.com
Name:   example.com
Address: 93.184.216.34
  
```

- Domain name is resolved to an IP address.
- You can now connect to the server.
- Multiple IPs may be returned (load balancing).

4. Inspect Different DNS Records

```

# A record (IPv4)
clouduser:~$ dig example.com A +short
93.184.216.34

# AAAA record (IPv6)
clouduser:~$ dig example.com AAAA +short
2606:2800:220:1:248:1893:25c8:1946

# CNAME record
clouduser:~$ dig www.example.com CNAME +short
www.example.com.cdnprovider.net.
  
```

- A record:** Maps a domain to an IPv4 address.
- AAAA record:** Maps a domain to an IPv6 address.
- CNAME record:** Maps a domain to another domain name (alias).

5. DNS Resolution Explained (Simple)



- DNS is like a phonebook for the internet.
- It converts human-readable names to IP addresses.
- Without DNS, you would need to remember IP addresses.

6. Common Failure Scenario



Application works by IP but not hostname.

- You can connect using the IP address (e.g. curl 93.184.216.34) but it fails with the domain name (e.g. curl example.com).
- This usually means a DNS issue, not a firewall or server issue.
- Check if the domain name resolves correctly.
- It could be a wrong record, missing record, or DNS propagation delay.

7. Cloud Connection



- Cloud providers (AWS Route 53, Azure DNS, Google Cloud DNS) manage DNS.
- Load balancers have DNS hostnames (e.g. alb-123.amazonaws.com).
- Instances may be accessed via private DNS inside a VPC.
- DNS issues can break access to your applications even if the servers are running.

8. Practice

- Run **dig**, **nslookup** and **host** for a domain (e.g. example.com).
- Check the A, AAAA and CNAME records.
- Find the IP address and try to connect using ping or curl.
- Investigate a domain that does not resolve (use a made-up name).



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

HTTP and API Troubleshooting with curl

Test. Inspect. Diagnose. Solve.

Learn
Practice
Build
Grow

1. Why curl is Indispensable



- Works everywhere (Linux, macOS, Windows).
- Helps you test websites, APIs and internal services.
- Shows you the real request and response.
- Useful for debugging, automation and monitoring.
- Essential for cloud engineers, DevOps and SRE.

2. Useful Commands

COMMAND	WHAT IT DOES
<code>curl <URL></code>	Makes a request and shows the response body.
<code>curl -I <URL></code>	Shows only the HTTP headers.
<code>curl -v <URL></code>	Shows detailed request and response (very useful for troubleshooting).

3. Example: Basic Request

```
clouduser:~$ curl https://example.com
<!DOCTYPE html>
<html>
<head><title>Example</title></head>
<body>
<h1>Welcome to Example.com</h1>
</body>
</html>
```

- Shows the response body from the server.
- Useful to confirm the service is reachable.

4. Example: View Headers

```
clouduser:~$ curl -I https://example.com
HTTP/1.1 200 OK
Date: Tue, 10 Sep 2024 12:00:00 GMT
Content-Type: text/html
Content-Length: 1256
Connection: keep-alive
Server: nginx
```

- Shows only the HTTP headers.
- Useful to check status code, server type, redirects, etc.

5. Example: Verbose Output (Debugging)

```
clouduser:~$ curl -v https://example.com
* Trying 93.184.216.34:443...
* Connected to example.com (93.184.216.34) port 443
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.0.1
> Accept: */*
< HTTP/1.1 200 OK
< Content-Type: text/html
< Content-Length: 1256
```

- Shows the full request and response.
- Helps you see DNS, connection, TLS and headers.

6. Common HTTP Status Codes

STATUS CODE	MEANING
200	OK – request succeeded.
301	Moved Permanently – redirected to a new URL.
401	Unauthorized – authentication required.
403	Forbidden – you don't have permission.
404	Not Found – the resource does not exist.
500	Internal Server Error – a server-side problem.

7. Test APIs and Health Endpoints

```
clouduser:~$ curl https://api.example.com/health
{"status":"healthy","version":"1.0.0"}
```

- Many applications expose a /health or /status endpoint.
- If this works, the application is running fine.

8. Real World Example

```
clouduser:~$ curl -I https://myapp.com/health
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 48
```

- Always test the /health endpoint first.
- If you get 200, the app is healthy.
- If it fails, check the application before blaming the load balancer, DNS or network.

9. Key Takeaways

- ✓ curl helps you test and debug quickly.
- ✓ Check status codes and headers.
- ✓ Use -v when you need details.
- ✓ Test health endpoints.
- ✓ Don't assume it's a load balancer or network issue.
- ✓ Follow the evidence.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Remote Servers with SSH

Connect. Secure. Transfer. Manage.

Learn
Practice
Build
Grow

1. How SSH Works



- You connect from your local machine to a remote server using SSH.
- The connection is encrypted.
- You authenticate using a username and a private key (or password).
- Once connected, you can run commands on the remote server as if you are sitting in front of it.

2. Useful Commands

COMMAND	WHAT IT DOES
<code>ssh user@server</code>	Connect to a remote server (using default key).
<code>ssh -i key.pem user@server</code>	Connect using a specific private key.
<code>scp file.txt user@server:/path</code>	Copy a file to a remote server.
<code>scp user@server:/path/file.txt .</code>	Copy a file from a remote server to your local machine.

3. Private and Public Keys



Private Key

- Stays on your local machine.
- Used to authenticate you.
- Keep it secret and secure.
- Example: key.pem



Public Key

- Placed on the remote server (in `~/.ssh/authorized_keys`).
- Allows the server to verify you.
- Can be shared (not sensitive).

4. Example: Connect to a Server

```

clouduser:~$ ssh -i key.pem ubuntu@54.210.12.34
The authenticity of host '54.210.12.34 (54.210.12.34)'
can't be established.
Are you sure you want to continue connecting (yes/no)? yes
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux x86_64)
  
```

- Replace key.pem with your private key file.
- Replace ubuntu with the correct username (e.g. ubuntu, ec2-user, centos).
- Replace 54.210.12.34 with your server's public IP or hostname.

5. Correct Private-Key Permissions

```

clouduser:~$ ls -l key.pem
-rw-r--r-- 1 clouduser clouduser 1675 Sep 10 10:00 key.pem
clouduser:~$ chmod 400 key.pem
clouduser:~$ ls -l key.pem
-r----- 1 clouduser clouduser 1675 Sep 10 10:00 key.pem
  
```

- Your private key should have restrictive permissions.
- Use `chmod 400 key.pem`.
- If permissions are too open, SSH may refuse to use the key (for security reasons).

6. Common Failures and Fixes

ERROR MESSAGE	CAUSE	SOLUTION
Connection timed out	Server unreachable (network, firewall).	Check IP, security group, network route.
Permission denied (publickey)	Wrong key, wrong permissions or key not in <code>authorized_keys</code> .	Use correct key, set <code>chmod 400</code> , check server's <code>authorized_keys</code> .
Please login as the user "xyz"	Wrong username.	Use the correct username (e.g. ubuntu, ec2-user, centos).

7. Security Reminder



- Never share your private keys.
- Keep your keys in a secure location.
- Use key-based authentication instead of passwords.
- Consider using a passphrase for additional security.
- If a key is compromised, delete it and create a new one.

8. Key Takeaways

- SSH allows you to securely connect to remote servers.
- Use `ssh` and `scp` for connection and file transfer.
- Understand the difference between private and public keys.
- Set correct permissions on your private key (`chmod 400`).
- Troubleshoot common errors: timeout, permission denied, wrong username.
- Always protect your private keys.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

JSON, YAML, and Command Output

Structured Data. Real Insights. Automation Ready.

1. Why Cloud CLIs Return Structured Data



- Cloud CLIs (AWS, Azure, GCP, Terraform, Kubernetes, etc.) return structured data (JSON or YAML).
- Structured data is machine-readable and easy to parse.
- It allows you to extract exactly what you need.
- It enables automation, scripting and integration with other tools.

3. Tools for Working with JSON and YAML

TOOL	WORKS WITH	PURPOSE	EXAMPLE
jq	JSON	Parse, filter and transform JSON data.	<code>jq '.[] .name'</code>
yq	YAML	Parse, filter and transform YAML data.	<code>yq '.items[].name'</code>

4. Pipes and Filtering

- Use pipes (`|`) to pass command output to another command.
- Filter the data to show only what you need.
- This saves time and makes output easier to read.
- Combine CLI commands with **jq** or **yq** to extract specific fields.

```
command | jq 'filter'
command | yq 'filter'
```

2. JSON vs YAML

JSON (JavaScript Object Notation)

- Format: Key-value pairs
- Syntax: Uses `{ }` and `[ ]`
- Commonly used by cloud CLIs and APIs
- More strict syntax
- Example:

```
{
  "name": "web-server",
  "type": "t2.micro",
  "status": "running"
}
```

YAML (YAML Ain't Markup Language)

- Format: Human-friendly
- Syntax: Uses indentation
- Commonly used for configuration files (Kubernetes, CI/CD, etc.)
- Easier to read and write
- Example:

```
name: web-server
type: t2.micro
status: running
```

5. Example: Resource List → jq → Selected Field

List EC2 instances and extract only the instance IDs (AWS CLI)

```
clouduser:~$ aws ec2 describe-instances \
  --query 'Reservations[].Instances[].InstanceId' \
  --output json | jq -r '[]'
i-0123456789abcdef0
i-0abcde1234567890f
i-0fedcba9876543210
```

- The AWS CLI returns JSON.
- jq** extracts only the instance IDs.
- You can filter for other fields like tags, state, or public IP.

6. Example: Working with YAML using yq

```
clouduser:~$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  image: nginx:1.25
```

```
clouduser:~$ yq '.metadata.name' deployment.yaml
my-app
```

```
clouduser:~$ yq '.spec.replicas' deployment.yaml
3
```

- yq** helps you read and extract values from YAML files.
- Useful for Kubernetes manifests, CI/CD configs and more.

7. DevOps Connection



- CLI output becomes input for automation scripts (Bash, Python, etc.).
- You can extract data and use it in pipelines (CI/CD).
- Structured data makes it easy to integrate with tools like Terraform, Ansible and Jenkins.
- Essential skill for DevOps, Cloud and SRE roles.

8. Key Takeaways

- Cloud CLIs return structured data (JSON or YAML).
- Use **jq** for JSON and **yq** for YAML.
- Pipes (`|`) help you filter and extract only the information you need.
- Structured output is designed for automation.
- This skill is essential for cloud engineers, DevOps and SRE.

9. Practice Exercises



- Run a cloud CLI command (e.g. `aws ec2 describe-instances`).
- Use **jq** to extract instance IDs, states and public IPs.
- Use **yq** to read a Kubernetes YAML file and extract the image name.
- Try filtering and formatting the output (e.g. use `jq -r`).
- Combine commands with pipes to get exactly the data you need.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta



LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Environment Variables and Credentials

Configure. Secure. Do Not Expose.

1. What are Environment Variables?



- Environment variables are key-value pairs that store configuration settings.
- They provide a way for applications and tools to access information.
- Common examples: PATH, AWS credentials, API keys, database URLs, and region settings.
- They help keep configuration separate from code.

2. Useful Commands

COMMAND	WHAT IT DOES
<code>env</code>	Shows all environment variables.
<code>printenv</code>	Shows environment variables (similar to <code>env</code>).
<code>export VAR=value</code>	Creates or sets an environment variable for the current session.
<code>unset VAR</code>	Removes (unsets) an environment variable.

3. Examples

```
clouduser:~$ env | grep PATH
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin

clouduser:~$ export PROJECT=veriqta
clouduser:~$ echo $PROJECT
veriqta

clouduser:~$ unset PROJECT
clouduser:~$ echo $PROJECT
```

- Use `echo $VAR` to check the value of a variable.
- Use `export` to set a variable for the current session.
- Use `unset` to remove a variable.

4. PATH Explained



- PATH is an environment variable that tells the shell where to look for executable files.
- It contains a list of directories separated by colons (:).
- Example: `/usr/local/bin:/usr/bin:/bin`
- When you run a command like `aws`, the shell searches these directories in order.
- You can add a directory to PATH using:
`export PATH=$PATH:/new/directory`

5. Environment Variables in Cloud Tooling



AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_REGION



AZURE_CLIENT_ID
AZURE_CLIENT_SECRET
AZURE_TENANT_ID
AZURE_SUBSCRIPTION_ID



GOOGLE_APPLICATION_CREDENTIALS
GOOGLE_CLOUD_PROJECT
GOOGLE_CLOUD_REGION

- Cloud CLIs and SDKs use environment variables to authenticate and configure access.
- You can set them in your shell, a `.env` file or CI/CD pipelines.
- They keep sensitive information out of your code.

6. Why Credentials Should Not Be Hardcoded



- Hardcoding credentials in code is a major security risk.
- If your code is shared (e.g. GitHub), anyone can see and use your credentials.
- It can lead to unauthorized access, data breaches and unexpected charges.
- Use environment variables, secret management tools (AWS Secrets Manager, Azure Key Vault, Google Secret Manager) or IAM roles instead.

7. Security Reminder



- Never expose secrets through repositories.
- Never share credentials in screenshots.
- Use environment variables or secret managers.
- Rotate credentials regularly.
- Follow the least privilege principle.

8. Key Takeaways

- ✓ Environment variables store configuration and sensitive data.
- ✓ Use `env`, `printenv`, `export` and `unset` to manage them.
- ✓ PATH tells the shell where to find executables.
- ✓ Cloud tools rely on environment variables for authentication.
- ✓ Never hardcode credentials in code.
- ✓ Always protect your secrets.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta



LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

AWS CLI Essentials

Same Power. Different Interface. Master Both.

1. Console vs AWS CLI

AWS Console



- Web-based interface.
- Click and manage resources.
- Great for learning, exploration and quick tasks.
- Visual and easy to use.
- May not be suitable for automation.

AWS CLI



- Command-line interface.
- Faster and more powerful for advanced users.
- Ideal for automation, scripting and DevOps.
- Works on Linux, macOS and Windows.
- Uses your AWS credentials, profiles and region.

2. Key Commands

COMMAND	WHAT IT DOES
<code>aws configure</code>	Sets up your AWS access key, secret key, region and output format.
<code>aws sts get-caller-identity</code>	Shows your AWS account ID, user/role and ARN (confirms who you are).
<code>aws ec2 describe-instances</code>	Lists your EC2 instances.
<code>aws s3 ls</code>	Lists your S3 buckets (or contents of a bucket).

3. Example: Configure AWS CLI

```
clouduser:~$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: *****
Default region name [None]: us-east-1
Default output format [None]: json
```

- This creates credentials in `~/.aws/credentials`.
- You can create multiple profiles for different accounts.
- Never share your access key or secret key.

4. Check Your Identity

```
clouduser:~$ aws sts get-caller-identity
{
  "UserId": "AIDA..EXAMPLE",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/clouduser"
}
```

- Shows your account ID, user/role and ARN.
- Always run this to confirm which account you are using.
- Very useful when you have multiple profiles.

5. List EC2 Instances

```
clouduser:~$ aws ec2 describe-instances

INSTANCE ID      INSTANCE TYPE  STATE  PUBLIC IP
i-0abc123def456  t3.micro       running 3.90.12.34
i-0def789ghi012  t3.small      stopped -
```

- Lists all EC2 instances in your configured region.
- You can add `--region` or `--profile` to specify.
- Use `--query` to filter and get only the information you need.

6. List S3 Buckets

```
clouduser:~$ aws s3 ls
2024-01-10 08:22:13 my-app-bucket
2024-03-05 14:11:01 veriqa-backups
2024-06-21 09:45:30 project-data
```

- Lists all S3 buckets in your account.
- Use `aws s3 ls s3://bucket-name` to list contents of a specific bucket.
- You can add `--region` if needed.

7. Profiles and Regions



- Profiles allow you to manage multiple AWS accounts (e.g. personal, work, production).
- Regions specify where your resources are located (e.g. us-east-1, eu-west-1).
- You can set a default profile and region, or override them using `--profile` and `--region` in any command.

8. Common Failure and How to Fix



- Command runs against the wrong profile or region.
- Resources not found (because you are in the wrong region).
- Access denied (insufficient permissions).
- Use `aws configure list` to check your current settings.
- Always confirm your account, identity and region before making changes.

9. Key Takeaways

- ✓ Understand the difference between the AWS Console and AWS CLI.
- ✓ Configure your CLI with the correct credentials, profile and region.
- ✓ Always verify your identity before making changes.
- ✓ Be careful with the profile and region you are using.
- ✓ Use the CLI to automate, script and manage your infrastructure safely.

10. DevOps Connection



- AWS CLI is essential for automation, CI/CD and IaC (Terraform, CloudFormation, etc.).
- You can use it in scripts, pipelines and monitoring tools.
- Knowing how to use the CLI is a core skill for DevOps, Cloud and SRE roles.

Azure CLI Essentials

Control. Automate. Build. With Azure CLI.

Learn
Practice
Build
Grow

1. How Azure CLI Communicates with Azure APIs



- Azure CLI (az) sends HTTPS requests to Azure REST APIs.
- You authenticate using your Azure account (user, service principal, or managed identity).
- The APIs return data (usually in JSON), which the CLI formats and displays.
- You can use CLI commands to manage all Azure resources (subscription, resource groups, VMs, storage, networking, etc.).

3. Example: Login and Check Subscription

```

clouduser:~$ az login
A browser will open for you to sign in...
Login successful.
clouduser:~$ az account show
{
  "name": "Visual Studio Enterprise",
  "id": "12345678-1234-1234-1234-123456789012",
  "tenantId": "abcd1234-abcd-1234-abcd-123456789012",
  "user": { "name": "user@company.com", "type": "user" }
}
  
```

- az login authenticates your account.
- az account show displays the current subscription details.
- You must be logged in before running other az commands.

2. Key Commands

COMMAND	WHAT IT DOES
az login	Authenticates you with Azure (opens a browser for sign-in).
az account show	Shows details of the current subscription.
az account list	Lists all subscriptions you have access to.
az group list	Lists all resource groups in the current subscription.
az vm list	Lists all virtual machines in the current subscription.

4. Subscription Context

- Your Azure account can have access to multiple subscriptions (e.g. personal, work, production).
- The CLI uses a default subscription for commands.
- You can change the active subscription using:

az account set --subscription <SUBSCRIPTION_ID>

- Always confirm which subscription you are using before making changes.



Use **az account list** to see all available subscriptions and **az account show** to confirm the current one.

5. Output Formats and Queries

Azure CLI returns output in JSON by default. You can format and query the output to get only the information you need.

EXAMPLE	DESCRIPTION
az vm list -o table	Displays output in a table format.
az vm list -o json	Displays output in JSON format (default).
az vm list -o tsv	Displays output as plain text (TSV).
az vm list --query "[].name]"	Returns only the VM names using a query.
az group list --query "[].name]" -o tsv	Lists only resource group names as text.

- Use --query with JMESPath syntax to filter and extract specific fields.
- This helps you get clean output for scripts and automation.

6. Example: List Resource Groups and VMs

```

clouduser:~$ az group list -o table
Name                Location
-----
dev-rg              eastus
prod-rg             westus
veriqta-rg          westeurope
clouduser:~$ az vm list --query "[].name,resourceGroup]" -o table
Name                ResourceGroup
-----
web-vm              dev-rg
db-vm               prod-rg
bastion-vm          veriqta-rg
  
```

- Use **-o table** for a readable format.
- Use **--query** to extract specific fields.
- These commands help you quickly see your resources.

7. Common Failure and How to Fix



- Command runs against the wrong subscription.
- Resources not found (because you are in the wrong subscription).
- Authentication errors (not logged in or token expired).
- Insufficient permissions.
- Fix: run **az account list**, set the correct subscription, and confirm with **az account show**.

8. Key Takeaways

- ✓ Azure CLI communicates with Azure APIs using authenticated requests.
- ✓ Use **az login** to authenticate.
- ✓ Always confirm your subscription before making changes.
- ✓ Use output formats (-o) and queries (--query) to get the information you need.
- ✓ Be careful when working with multiple subscriptions.
- ✓ Azure CLI is essential for automation, scripting and infrastructure management.
- ✓ Always double-check before creating, deleting or modifying resources.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta



LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Google Cloud CLI Essentials

Authenticate. Configure. Manage. Automate.

1. Understanding the gcloud CLI



Google Cloud

One CLI.
Full control
of your cloud.

- The gcloud CLI is the command-line tool for managing Google Cloud resources.
- It communicates with Google Cloud APIs over HTTPS.
- You authenticate with your Google account or a service account.
- It allows you to create, configure and manage resources (VMs, storage, networking, etc.).
- It returns output in human-readable or JSON format.

2. Key Commands

COMMAND	WHAT IT DOES
<code>gcloud auth login</code>	Authenticates you with Google Cloud (opens a browser for sign-in).
<code>gcloud config list</code>	Shows your current CLI configuration (account, project, region, etc.).
<code>gcloud projects list</code>	Lists all projects you have access to.
<code>gcloud compute instances list</code>	Lists all Compute Engine VM instances in the current project.

3. Example: Login and Check Configuration

```
clouduser:~$ gcloud auth login
Your browser has been opened to visit:
https://accounts.google.com/o/oauth2/auth?...
You are now logged in as [your-email@gmail.com].

clouduser:~$ gcloud config list
[core]
account = your-email@gmail.com
project = my-project-123456
compute/region = us-central1
compute/zone = us-central1-a
```

- Use `gcloud auth login` to authenticate.
- Use `gcloud config list` to check your current settings.
- Make sure the correct account and project are selected.

4. Project and Region/Zone Context



Project
(Organizes resources)



Region
(e.g. us-central1)



Zone
(e.g. us-central1-a)

- A project is a container for all your Google Cloud resources.
- A region is a geographic location (e.g. us-central1, europe-west1).
- A zone is a specific data center within a region (e.g. us-central1-a).
- You can set the default project, region and zone using:

```
gcloud config set project PROJECT_ID
gcloud config set compute/region REGION
gcloud config set compute/zone ZONE
```

5. Authentication vs Authorization



Authentication

- Proves who you are (e.g. Google account or service account).
- Done with: `gcloud auth login` or service account key.
- Without authentication, you cannot access Google Cloud.



Authorization

- Determines what you can do (permissions and roles).
- Controlled by IAM (Identity and Access Management).
- Even if you are authenticated, you may not have permission to perform certain actions.

6. Common Failure: Correct Login, Wrong Project

```
clouduser:~$ gcloud compute instances list
Listed 0 items.
clouduser:~$ gcloud config get-value project
wrong-project-999
clouduser:~$ gcloud projects list
PROJECT_ID  NAME
my-project-123456  My Production Project
dev-project-789    Development Project
test-project-001   Test Project
```

- You are logged in, but the wrong project is selected.
- Always check your project with `gcloud config get-value project`.
- Use `gcloud projects list` to see all accessible projects.
- Set the correct project before running other commands.

7. Key Takeaways

- ✓ The gcloud CLI helps you manage Google Cloud resources.
- ✓ Authenticate with the correct account (user or service account).
- ✓ Always verify your project, region and zone before making changes.
- ✓ Understand the difference between authentication and authorization.
- ✓ Use CLI commands carefully to avoid changes in the wrong project.
- ✓ When in doubt, run `gcloud config list` and `gcloud projects list`.

8. DevOps Connection



- The gcloud CLI is essential for automation, CI/CD and IaC (Terraform, Deployment Manager, etc.).
- It is used in scripts, pipelines and monitoring tools.
- Service accounts are commonly used in automated environments.
- Correct project and permissions are critical in shared environments.
- Always follow the principle of least privilege.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Cloud Storage from the Command Line

Same Concept. Different Cloud. Real Skills.

1. Object Storage Mental Model



- Object storage stores data as objects (files) in buckets/containers.
- You interact with it using a CLI, SDK or API.
- It is highly scalable, durable and accessible from anywhere.
- Common uses: backups, logs, static websites, data for analytics, application files and more.

3. Common Operations

OPERATION	AWS S3	Azure Storage	Google Cloud Storage
List objects	<code>aws s3 ls s3://bucket/</code>	<code>az storage blob list --container-name container</code>	<code>gsutil ls gs://bucket/</code>
Upload file	<code>aws s3 cp file.txt s3://bucket/</code>	<code>az storage blob upload --container-name container --file file.txt --name file.txt</code>	<code>gsutil cp file.txt gs://bucket/</code>
Download file	<code>aws s3 cp s3://bucket/file.txt file.txt</code>	<code>az storage blob download --container-name container --name file.txt --file file.txt</code>	<code>gsutil cp gs://bucket/file.txt file.txt</code>
Inspect object	<code>aws s3api head-object --bucket bucket --key file.txt</code>	<code>az storage blob show --container-name container --name file.txt</code>	<code>gsutil stat gs://bucket/file.txt</code>

2. CLI Examples by Cloud Provider



```
# AWS S3 (using AWS CLI)
aws s3 ls
aws s3 cp file.txt s3://my-bucket/
aws s3 cp s3://my-bucket/file.txt .
aws s3 ls s3://my-bucket/
```



```
# Azure Storage (using Azure CLI)
az storage blob list --container-name mycontainer
--account-name mystorageaccount
az storage blob upload --container-name mycontainer
--file file.txt --name file.txt
az storage blob download --container-name mycontainer
--name file.txt --file file.txt
```



Google Cloud

```
# Google Cloud Storage (using gcloud CLI)
gsutil ls
gsutil cp file.txt gs://my-bucket/
gsutil cp gs://my-bucket/file.txt .
gsutil ls gs://my-bucket/
```

4. What Can Go Wrong?



- Wrong path or bucket/container name (e.g. `s3://mybucket` vs `s3://my-bucket`).
- Permissions and access denied (IAM roles, service accounts, or ACLs).
- Wrong region, account or project.
- Typo in file names or paths.
- Trying to access a private object without proper authentication.
- Large files timing out or incomplete uploads.

5. Example: Upload and Download a File (AWS S3)

```
clouduser:~$ echo "Hello Cloud" > demo.txt
clouduser:~$ aws s3 cp demo.txt s3://veriqta-demo/
upload: ./demo.txt to s3://veriqta-demo/demo.txt
clouduser:~$ aws s3 ls s3://veriqta-demo/
2024-09-14 10:22:30      12 demo.txt
clouduser:~$ aws s3 cp s3://veriqta-demo/demo.txt .
download: s3://veriqta-demo/demo.txt to ./demo.txt
```

Same flow applies to Azure and GCP:

- List objects
- Upload file
- Download file
- Inspect object

6. Production Awareness



- Verify the path, bucket/container, and account/project before running recursive commands (e.g. `--recursive`).
- Double-check before deleting or synchronizing.
- Use `--dry-run` or list commands to confirm what will be affected.
- Be careful with automated scripts and CI/CD pipelines.
- Enable versioning (where available) to protect against accidental deletion.

7. Key Takeaways

- Understand the object storage mental model.
- Learn the basic CLI commands for AWS S3, Azure Storage and Google Cloud Storage.
- Always verify your bucket/container, path, region and account/project.
- Use CLI commands to list, upload, download and inspect objects.
- Be careful with permissions and access controls.
- Always verify before running recursive deletion or synchronization.

8. DevOps Connection



- Object storage is used for backups, logs, artifacts, static websites and data pipelines.
- You will work with it in automation (Terraform, CI/CD, backup scripts, etc.).
- Proper access control and verification are critical in production environments.
- It is a core skill for DevOps, Cloud and SRE roles.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Cloud CLI Troubleshooting Workflow

A Simple. Reusable. Logical Method. Works on Any Cloud.

Learn
Practice
Build
Grow

1. The Troubleshooting Workflow

- 1  **SYMPTOM** What is the problem?
What are you seeing?
- 2  **IDENTITY** Who am I?
Am I using the correct account?
- 3  **CONTEXT** What region, project or environment
am I working in?
- 4  **RESOURCE STATE** Is the resource running and healthy?
- 5  **NETWORK** Can I reach it? Is the network working?
- 6  **PERMISSIONS** Do I have the required permissions?
- 7  **LOGS** What do the logs tell me?
- 8  **FIX** Make the right change.
- 9  **VERIFY** Confirm that the issue is resolved.

4. Example Troubleshooting Flow

```
# 1. Check DNS resolution
clouduser:~$ dig myapp.example.com

# 2. Test connectivity
clouduser:~$ curl -v https://myapp.example.com

# 3. Check local network routes
clouduser:~$ ip route

# 4. Check open ports
clouduser:~$ ss -tulnp | grep :443

# 5. Verify cloud resource (AWS example)
clouduser:~$ aws ec2 describe-instances --filters \
  Name=tag:Name,Values=myapp"

# 6. Check logs (AWS example)
clouduser:~$ aws logs tail /aws/ec2/myapp --follow
```

6. Junior Lesson



Never randomly
change five things
at once.

- Follow a logical process.
- Change one thing at a time.
- Understand what you are testing.
- Document what you find.
- This builds real engineering skills.

2. Scenario: "I Cannot Reach My Cloud Application."



Symptom: I cannot reach my cloud application.

- The application URL is not responding.
- It works for others, but not for me.
- I need to find out where the problem is.

3. Useful Commands for Each Step

STEP	COMMANDS	PURPOSE
Identity	whoami aws sts get-caller-identity az account show gcloud auth list	Verify your current user and cloud account.
Context	aws configure list az account list gcloud config list	Check the current region, project and settings.
Network	ip route dig <domain> curl -v <url> ss -tulnp	Check local network, DNS, connectivity and open ports.
Resource State	aws ec2 describe-instances az vm list gcloud compute instances list	Verify the resource is running and healthy.
Permissions	aws iam get-user az role assignment list gcloud projects get-iam-policy	Confirm you have the required permissions.
Logs	aws logs tail <log-group> az monitor log-analytics query gcloud logging read	Check application and infrastructure logs.

5. Common Issues and Fixes

ISSUE	POSSIBLE CAUSE	FIX
DNS not resolving	Wrong hostname or DNS configuration	Check DNS records, use dig
Connection timeout	Security group / firewall blocking traffic	Verify network rules and ports
Permission denied	IAM role or user lacks access	Update permissions
Resource not found	Wrong region or project	Check and set the correct region
Application error	Service not running or unhealthy	Check resource state and logs

7. Key Takeaways

- ✓ Use a structured troubleshooting workflow.
- ✓ Always verify your identity, context and permissions.
- ✓ Check network connectivity and DNS.
- ✓ Inspect the resource state and logs.
- ✓ Fix the real cause, not just the symptom.
- ✓ Verify the solution.
- ✓ This method works on AWS, Azure, Google Cloud and every cloud environment.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Dangerous Commands and Production Safety

Be Powerful. Be Careful. Think Before You Run.

1. rm -rf (Be Very Careful)

```
clouduser:~$ rm -rf /important-folder
# This permanently deletes files without asking.
```

- rm -rf deletes files and directories recursively.
- There is no recycle bin.
- Be extra careful with paths (e.g. / vs /home/user).
- Always double-check before running.

2. Broad chmod (Dangerous Permissions)

```
clouduser:~$ chmod 777 script.sh
# Gives full permissions to everyone.
```

- Broad permissions (e.g. 777) can expose sensitive files.
- Only give the minimum permissions needed (e.g. 644 or 750).
- Be careful when changing permissions recursively (chmod -R).

3. sudo (Use Responsibly)

```
clouduser:~$ sudo command
# Runs command as root (administrator).
```

- sudo gives elevated privileges.
- A mistake with sudo can damage your system.
- Only use sudo when necessary.
- Understand what the command does before running it.

4. Cloud Delete/Terminate Commands

 `aws ec2 terminate-instances --instance-ids i-1234567890`

 `az vm delete --name myvm --resource-group myrg`

 `gcloud compute instances delete my-vm --zone us-central1-a`

- These commands permanently delete resources.
- Always verify the resource ID, name and environment before running.
- Use filters and tags to confirm you are targeting the correct resource.

5. Recursive Storage Operations

 `aws s3 rm s3://my-bucket/ --recursive`

 `az storage blob delete-batch --source mycontainer`

 `gsutil -m rm -r gs://my-bucket/`

- Recursive commands can delete large amounts of data.
- Always verify the bucket/container name and path.
- Consider using `--dry-run` or list commands first.
- Be careful with `--recursive` on production data.

6. Wrong Account, Region or Project

```
# Example: Deleting in the wrong account
aws s3 rm s3://company-backups --recursive

# Example: Wrong region
az vm delete --name myvm --resource-group myrg --region eastus

# Example: Wrong project
gcloud compute instances delete my-vm --zone us-central1-a --project wrong-project
```

- Make sure you are using the correct account, subscription or project.
- Verify the correct region or zone.
- Use CLI commands to confirm your current context (e.g. `aws sts get-caller-identity`, `az account show`, `gcloud config list`).

7. Production vs Development Context

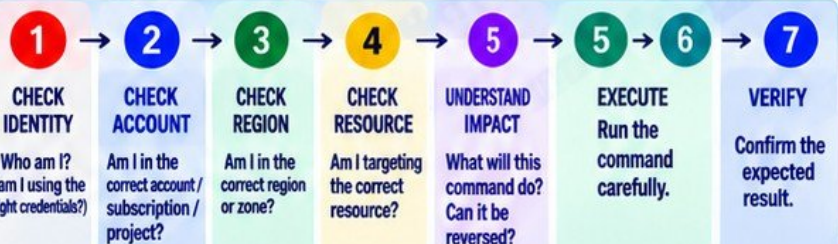
Production

- Real users and real data.
- Changes can cause downtime and financial loss.
- Be extra careful. Follow change management processes.
- Use approvals where possible.

Development

- Used for testing and learning.
- You can experiment more freely.
- Still be careful, especially with shared environments.
- Use clear naming to avoid confusion (e.g. dev-, test-).

8. Safe Command Execution Workflow



9. Production Awareness



- Command-line speed increases both your power and your ability to make mistakes.
- Always think before you run.
- Use dry-run options where available.
- Double-check everything, especially in production.
- A small mistake can have a big impact.

10. Key Takeaways

- Understand what every command does before running it.
- Always verify identity, account, region, resource and impact.
- Use dry-run or list commands first.
- Be careful with destructive commands (rm -rf, delete, terminate).
- Use least privileges and avoid broad permissions.
- Separate production and development environments clearly.
- Never make multiple changes at once.
- Build the habit of being careful. It will save you in the real world.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures

Final Cloud Engineer Command-Line Lab

A Realistic Scenario. A Complete Investigation. Real Skills.

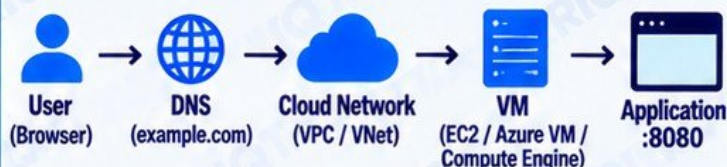
1. Scenario



"The application is deployed, but users cannot reach it."

- Users report that the application is not accessible.
- The application is running on port 8080.
- You are the junior cloud engineer on call.
- Use the command-line toolkit to investigate, find the root cause, fix it and verify.

2. Architecture



3. Investigation Steps and Commands

STEP	WHAT TO CHECK	COMMAND(S)
1	DNS resolution Is the domain resolving to the correct IP?	dig example.com
2	Network connectivity Can you reach the server?	ping <server-ip> tracert <server-ip>
3	HTTP connectivity Is the application responding?	curl -v http://<server-ip>:8080 curl -v http://<example.com>:8080
4	Listening ports Is the application listening on port 8080?	ss -tulpn grep :8080
5	Running processes Is the application process running?	ps aux grep <app-name>
6	Service state Is the service healthy?	systemctl status <service-name>
7	Logs What errors are showing?	journalctl -u <service-name> -n 50
8	Cloud identity Am I using the correct account?	aws sts get-caller-identity az account show gcloud auth list
9	VM / resource state Is the VM running and healthy?	aws ec2 describe-instances az vm list gcloud compute instances list
10	Cloud networking Are security groups, NSGs or firewalls allowing traffic?	aws ec2 describe-security-groups az network nsg list gcloud compute firewall-rules list
11	Permissions Do I have the required access?	Check IAM roles, policies and permissions for the resources.

4. Example Investigation (Terminal Output)

```

clouduser:~$ dig example.com
;; ANSWER SECTION:
example.com.      300  IN  A  203.0.113.10

clouduser:~$ curl -v http://203.0.113.10:8080
* Connected to 203.0.113.10 (203.0.113.10) port 8080
> GET / HTTP/1.1
> Host: 203.0.113.10:8080
< HTTP/1.1 200 OK
< Content-Type: text/html
<
... application is running ...

clouduser:~$ ss -tulpn | grep :8080
LISTEN  0      128          0.0.0.0:8080      0.0.0.0:*
users:((("python",pid=1234,fd=3))
  
```

5. Find the Root Cause and Fix It

- Analyse the evidence from each layer.
- Identify the failing component (e.g. security group blocking port 8080).
- Apply the correct fix (e.g. allow inbound traffic on port 8080).
- Re-test the connectivity and application.
- Verify the application works from a browser or curl.

6. Common Root Causes

- DNS pointing to the wrong IP.
- Security group / NSG / firewall blocking traffic.
- Application not running or listening on the expected port.
- Wrong server, region or account.
- Service crash or misconfiguration.
- Incorrect permissions.

7. Final Lesson

A cloud engineer does not guess.



- Observe.
- Collect evidence.
- Isolate the failing layer.
- Fix the cause.
- Verify the result.

Use a logical, structured approach. The same method works on AWS, Azure, GCP and any cloud environment.

8. Key Takeaways

- Follow a step-by-step investigation process.
- Use the right command for each layer.
- Always verify identity, account, region and permissions.
- Do not make multiple changes at once.
- Document what you find and how you fixed it.
- Practice this workflow until you can do it confidently.
- These skills are essential for real-world cloud engineering.



@veriqta



@veriqta



veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Futures